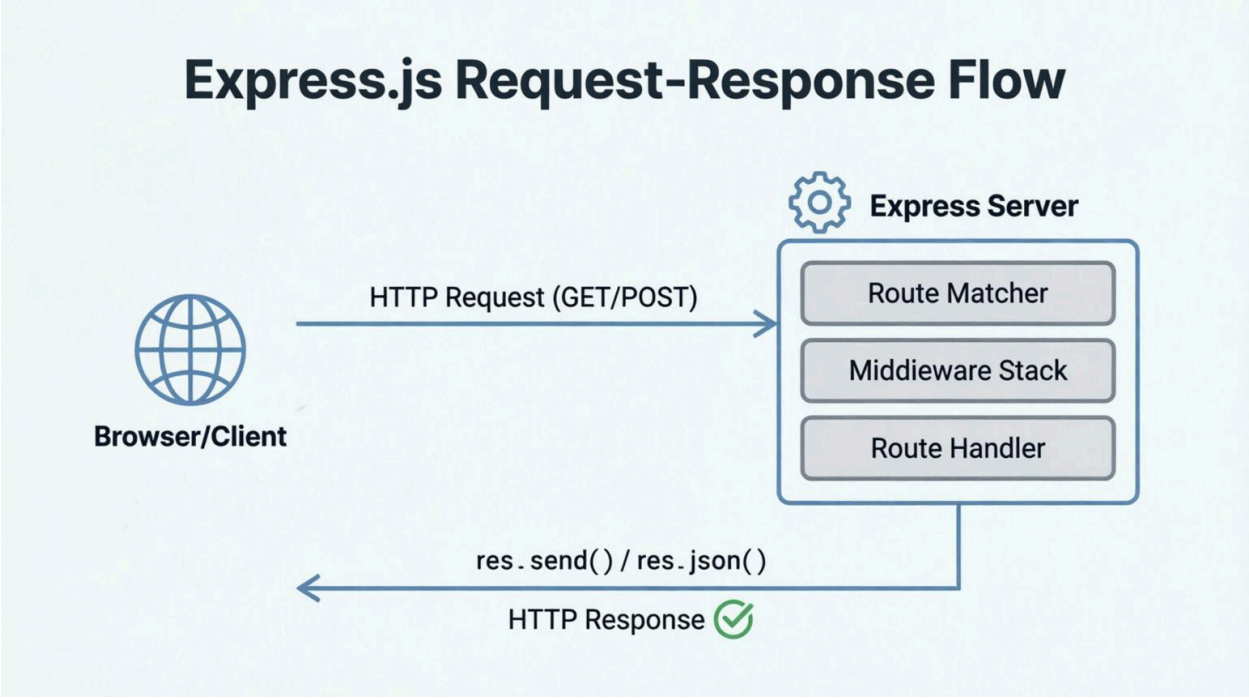
1. Introduction to Express.js



Express.js is a lightweight web application framework built on top of Node.js. It simplifies the development of web servers and APIs by providing convenient methods for routing, middleware processing, request handling, and response generation.

Node.js provides the runtime environment, while Express.js provides a structured way to build web applications using that runtime.

Why do we use Express.js?

- Simplifies server creation.
- Provides routing for different URLs.
- Handles HTTP requests and responses.
- Supports middleware for authentication, logging, and validation.
- Serves static files.
- Supports template engines such as EJS and Pug.
- Helps develop REST APIs and full-stack web applications.

Basic architecture

Client / Browser

Sends an HTTP request

Express.js Server

Routing → Middleware → Request handler

HTTP Response

HTML, JSON, text, or other data

Installation and setup

Prerequisite: Install Node.js (preferably a current LTS version) from nodejs.org.

Open a terminal and run:

```
mkdir expressdemo
cd expressdemo
npm init -y
npm install express
```

Create a file named app.js.

Program 1: First Express server

```
const express = require('express');

const app = express();
const PORT = 3000;

// Define a route
app.get('/', (req, res) => {
   res.send('Hello, B.Tech Students! Welcome to Express.js.');
});
```

```
// Start the server
app.listen(PORT, () => {
   console.log(`Server running at http://localhost:${PORT}`);
});
```

How to execute

Run this command inside the project directory:

node app.js

Expected terminal output:

Server running at http://localhost:3000

Open http://localhost:3000 in your browser.

Expected browser output:

Hello, B.Tech Students! Welcome to Express.js

- require('express') imports the Express framework.
- express() creates an Express application.
- app.get() registers a handler for GET requests to /.
- req represents the incoming request.
- res represents the outgoing response.
- res.send() sends the response to the client.
- app.listen() starts the server on port 3000.

Routing in Express.js

Routing determines how a server responds when a client requests a particular URL using a specific HTTP method.

For example:

HTTP method	URL	Purpose
GET	/	Home page
GET	/about	About page
GET	/students	Display students
GET	/students/101	Display a particular student
POST	/students	Submit student information

The general syntax is:

app.METHOD(PATH, HANDLER);

Here, METHOD is an HTTP method such as get or post, PATH is the route URL, and HANDLER is the callback function that processes the request.

Multiple routes and route parameters

```
const express = require('express');
const app = express();

app.get('/', (req, res) => {
   res.send('Home Page');
});
```

```
app.get('/about', (req, res) => {
   res.send('About the College');
});

app.get('/students/:id', (req, res) => {
   const studentId = req.params.id;
   res.send(`Student ID: ${studentId}`);
});

app.listen(3000, () => {
   console.log('Server started on port 3000');
});
```

Run:

node app.js

Test these URLs in your browser:

- http://localhost:3000/ → Home Page
- http://localhost:3000/about → About the College
- http://localhost:3000/students/101 → Student ID: 101

Explanation of route parameters

In the route /students/:id, :id is a route parameter. Express stores the matched value in req.params.id.

For /students/101, the value is "101" (a string).

Query parameters

Query parameters are values supplied after ? in a URL.

For example:

http://localhost:3000/search?name=Ravi&branch=CSE

Add this route to the same program:

```
app.get('/search', (req, res) => {
   const name = req.query.name;
   const branch = req.query.branch;

   res.send(`Name: ${name}, Branch: ${branch}`);
});
```

Expected output:

Name: Ravi, Branch: CSE

Difference: req.params retrieves values from the route path, whereas req.query retrieves values from the query string.

Handling GET and POST requests

HTTP methods specify the type of operation a client wants to perform.

GET request

- Used to retrieve information from the server.
- Parameters may appear in the URL.
- Commonly used for search results and displaying records.
- Should not be used to perform state-changing operations.

POST request

- Used to submit data to the server.
- Data is commonly sent in the request body.
- Used for registration forms, login forms, and creating records.
- Requires suitable body-parsing middleware to read form or JSON data.

Important Express methods

Method	Purpose
app.get()	Handles GET requests
app.post()	Handles POST requests
express.urlencoded()	Parses URL-encoded form data
express.json()	Parses JSON request bodies
res.send()	Sends a response
res.json()	Sends a JSON response

GET and POST with a student registration form

Create app.js:

```
const express = require('express');
const app = express();

// Middleware to read HTML form data
app.use(express.urlencoded({ extended: false }));

// GET route: display the form
app.get('/register', (req, res) => {
```

```
    res.send(`
        <h2>Student Registration</h2>
        <form method="POST" action="/register">
            <label>Name:</label>
            <input type="text" name="name" required>
            <br><br>

            <label>Branch:</label>
            <input type="text" name="branch" required>
            <br><br>

            <button type="submit">Register</button>
        </form>
    `);
});

// POST route: process submitted data
app.post('/register', (req, res) => {
    const name = req.body.name;
    const branch = req.body.branch;

    res.send(`
        <h2>Registration Successful</h2>
        <p>Name: ${name}</p>
        <p>Branch: ${branch}</p>
    `);
});

app.listen(3000, () => {
    console.log('Server running at http://localhost:3000');
});
```

Execution

1. Save the code in app.js.

2. Run node app.js.
3. Open http://localhost:3000/register.
4. Enter Ravi as the name and CSE as the branch.
5. Click Register.

Expected response:

Registration Successful
Name: Ravi
Branch: CSE

How the program works

GET /register

Server returns the HTML form

User enters name and branch

Browser submits the form

POST /register

express.urlencoded() parses the form body

Response

Server displays submitted information

Important: req.body contains client-supplied input. Validate it before using it

Middleware functions in Express.js

Middleware is a function that executes during the request-response cycle. It can inspect or modify the request, perform an operation, send a response, or pass control to the next middleware using next().

A middleware function generally has this form:

function middlewareName(req, res, next) {

```
  // Perform an operation
  next();
}
```

The three parameters are:

- req: Incoming request object.
- res: Response object.
- next: Function that transfers control to the next middleware or route handler.

Types of middleware

Type	Example	Purpose
Application-level	app.use()	Applied to the application or a path
Router-level	router.use()	Applied to an Express router
Built-in	express.json()	Parses JSON request bodies
Third-party	morgan	Logs HTTP requests
Custom	User-defined function	Implements application-specific logic
Error-handling	(err, req, res, next)	Handles errors

Custom logging middleware

```
const express = require('express');
const app = express();

// Custom middleware
function requestLogger(req, res, next) {
   console.log(
      `${new Date().toISOString()} - ${req.method} ${req.url}`
   );

   next();
}

// Register middleware
app.use(requestLogger);

app.get('/', (req, res) => {
   res.send('Home Page');
});

app.get('/about', (req, res) => {
   res.send('About Page');
});

app.listen(3000, () => {
   console.log('Server running on port 3000');
});
```

Execution

Run node app.js, then visit:

- http://localhost:3000/
- http://localhost:3000/about

Expected terminal output will resemble:

Server running on port 3000
2026-10-13T10:30:00.000Z - GET /
2026-10-13T10:30:05.000Z - GET /about

The timestamps are illustrative; actual timestamps will vary.

1. A request reaches the Express application.
2. app.use(requestLogger) executes the logging middleware.
3. The middleware prints the HTTP method and requested URL.
4. next() transfers control to the matching route.
5. The route sends a response.

What happens if next() is omitted?

If middleware neither sends a response nor calls next(), the request will generally remain pending. This is a common mistake when writing custom middleware.

Third-party middleware example: Morgan

Morgan is an HTTP request logger.

Install it:

npm install morgan

Use it in your application:

```
const express = require('express');
const morgan = require('morgan');

const app = express();

app.use(morgan('dev'));

app.get('/', (req, res) => {
   res.send('Morgan logging example');
```

```
});

app.listen(3000, () => {
   console.log('Server running on port 3000');
});
```

When you visit /, Morgan prints a log entry containing the request method, URL, status code, and response time.

Creating and using custom middleware

Custom middleware allows developers to define their own request-processing logic. It is useful for:

- Checking authentication.
- Validating incoming data.
- Logging requests.
- Checking permissions.
- Adding common information to request objects.
- Controlling access to particular routes.

Middleware may be applied globally with app.use() or selectively to particular routes.

Custom access-check middleware

This program demonstrates a simple request header check.

```
const express = require('express');

const app = express();

// Custom middleware
function checkAccess(req, res, next) {
   const accessKey = req.get('x-access-key');

   if (accessKey !== 'student123') {
```

```
        return res.status(401).json({
            message: 'Access denied'
        });
    }

    next();
}

// Public route
app.get('/', (req, res) => {
    res.send('Welcome to the student portal');
});

// Protected route
app.get('/dashboard', checkAccess, (req, res) => {
    res.send('Welcome to the student dashboard');
});

app.listen(3000, () => {
    console.log('Server running on port 3000');
});
```

Test the program

First, open http://localhost:3000/dashboard in a browser.

Expected JSON response:

```
{
  "message": "Access denied"
}
```

Now use a terminal to send the required header:

```
curl -H "x-access-key: student123" http://localhost:3000/dashboard
```

Expected output:

Welcome to the student dashboard

- req.get('x-access-key') reads the request header.
- If the value is incorrect, the middleware returns an HTTP 401 Unauthorized response.
- return prevents further execution after sending the response.
- If the value is correct, next() passes control to the dashboard route.

Global vs route-specific middleware

Global middleware:

app.use(requestLogger);

This runs for requests reaching the application after the middleware is registered, subject to its mount path.

Route-specific middleware:

app.get('/dashboard', checkAccess, dashboardHandler);

This runs the access check for the dashboard route only.

Security note: Hard-coded keys are not suitable for production. Real applications should use proper authentication, securely stored secrets, and authorization checks.

Serving static files in Express.js

Static files are files that the server delivers directly to the client without generating their contents dynamically for each request.

Examples include:

- HTML files
- CSS stylesheets

- Client-side JavaScript files
- Images and icons
- Fonts and other assets

Express provides the built-in middleware express.static() to serve these files.

Syntax:

app.use(express.static('public'));

Here, public is the directory containing the files to be served. A file named style.css inside that directory becomes available at /style.css.

Serve an HTML page, CSS, and an image

Create the following directory structure:

express-learning/

app.js

public/

index.html

style.css

college.jpg (optional image)

File 1: app.js

```
const express = require('express');
const path = require('path');

const app = express();

// Serve files from the public directory
app.use(express.static(path.join(__dirname, 'public')));
```

```
app.listen(3000, () => {
   console.log('Server running at http://localhost:3000');
});
```

File 2: public/index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
   <meta charset="UTF-8">
   <title>My College</title>
   <link rel="stylesheet" href="/style.css">
</head>
<body>
   <h1>Welcome to My College</h1>
   <p>This page is served using Express.js.</p>

   <img src="/college.jpg"
        alt="College campus"
        width="300">
</body>
</html>
```

File 3: public/style.css

```
body {
   font-family: Arial, sans-serif;
   background-color: #eef4ff;
   text-align: center;
   padding: 40px;
}

h1 {
```

```
    color: #174ea6;
}

p {
    font-size: 18px;
    color: #333333;
}
```

Place an image named college.jpg in public if you want the image to display. Otherwise, remove the <img> element.

Execution

Run:

node app.js

Open:

http://localhost:3000/

Expected result: A styled college webpage with a heading, paragraph, and optional image.

- express.static() serves files from the specified directory.
- path.join(__dirname, 'public') creates an absolute directory path.
- __dirname refers to the directory containing app.js in CommonJS modules.
- The browser automatically requests /style.css and /college.jpg as it loads the HTML.

Important: Static files do not need individual Express routes. Express automatically serves matching files from the configured directory.

Templating engines: EJS and Pug

A templating engine allows the server to generate HTML dynamically by combining an HTML template with data.

For example, a student dashboard may display a different student name, marks, or list of subjects depending on the data received from the server.

Express supports multiple template engines, including:

- EJS (Embedded JavaScript): Uses HTML with special tags for JavaScript expressions and logic.
- Pug: Uses indentation-based syntax to describe HTML elements.

How template rendering works

Express route

Prepares data, such as student name and marks

Template engine

Combines data with EJS or Pug template

Generated HTML

Sent to the browser as the response

Program using EJS

Step 1: Install EJS

npm install ejs

Step 2: Create the directory structure

```
express-learning/
├── app.js
└── views/
    └── student.ejs
```

Step 3: Create app.js

```
const express = require('express');
const app = express();
```

```
// Configure EJS as the template engine
app.set('view engine', 'ejs');

app.get('/student', (req, res) => {
    const student = {
        name: 'Ravi',
        branch: 'CSE',
        marks: 85,
        subjects: ['DBMS', 'OS', 'Computer Networks']
    };

    res.render('student', { student });
});

app.listen(3000, () => {
    console.log('Server running on port 3000');
});
```

Step 4: Create views/student.ejs

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>Student Details</title>
</head>
<body>
    <h1>Student Details</h1>

    <p>Name: <%= student.name %></p>
    <p>Branch: <%= student.branch %></p>
    <p>Marks: <%= student.marks %></p>

    <h2>Subjects</h2>
    <ul>
```

```
    <% student.subjects.forEach(function(subject) { %>
      <li><%= subject %></li>
    <% }); %>
  </ul>

  <% if (student.marks >= 40) { %>
    <p>Result: Pass</p>
  <% } else { %>
    <p>Result: Fail</p>
  <% } %>
</body>
</html>
```

Step 5: Execute

node app.js

Open http://localhost:3000/student.

Expected webpage content:

Student Details

Name: Ravi

Branch: CSE

Marks: 85

Subjects

- DBMS
- OS
- Computer Networks

Result: Pass

Important EJS tags

Tag	Meaning
<%= value %>	Outputs a value with HTML escaping
<%- value %>	Outputs unescaped content
<% code %>	Executes JavaScript without directly outputting it

For example:

<h1><%= student.name %></h1>

This displays the student's name while escaping HTML special characters. Avoid unescaped output with <%- ... %> for untrusted data because it can introduce cross-site scripting (XSS) vulnerabilities.

Program using Pug

Pug is an alternative template engine. Unlike EJS, Pug uses indentation instead of conventional HTML opening and closing tags.

Step 1: Install Pug

npm install pug

Step 2: Create the directory structure

express-learning/
├── app.js
└── views/
    └── student.pug

Step 3: Create app.js

```
const express = require('express');
const app = express();

app.set('view engine', 'pug');

app.get('/student', (req, res) => {
    const student = {
        name: 'Anita',
        branch: 'IT',
        marks: 92,
        subjects: ['Java', 'Python', 'Web Technology']
    };

    res.render('student', { student });
});

app.listen(3000, () => {
    console.log('Server running on port 3000');
});
```

Step 4: Create views/student.pug

```
doctype html
html
  head
    title Student Details
  body
    h1 Student Details
    p Name: #{student.name}
    p Branch: #{student.branch}
    p Marks: #{student.marks}

    h2 Subjects
```

```
  ul
    each subject in student.subjects
      li= subject

  if student.marks >= 40
    p Result: Pass
  else
    p Result: Fail
```

Step 5: Execute

node app.js

Open http://localhost:3000/student.

Expected output:

- Name: Anita
- Branch: IT
- Marks: 92
- Subjects: Java, Python, Web Technology
- Result: Pass

EJS vs Pug

Feature	EJS	Pug
Syntax	HTML with JavaScript tags	Indentation-based
HTML tags	Written explicitly	Inferred from tag names
Learning curve	Easier for students familiar with HTML	Requires learning Pug syntax

Logic	JavaScript inside template tags	Indentation-based conditions and loops
File extension	.ejs	.pug

1. Introduction to NoSQL Databases

What is a database?

A database is an organized collection of data that can be stored, accessed, managed, and updated efficiently.

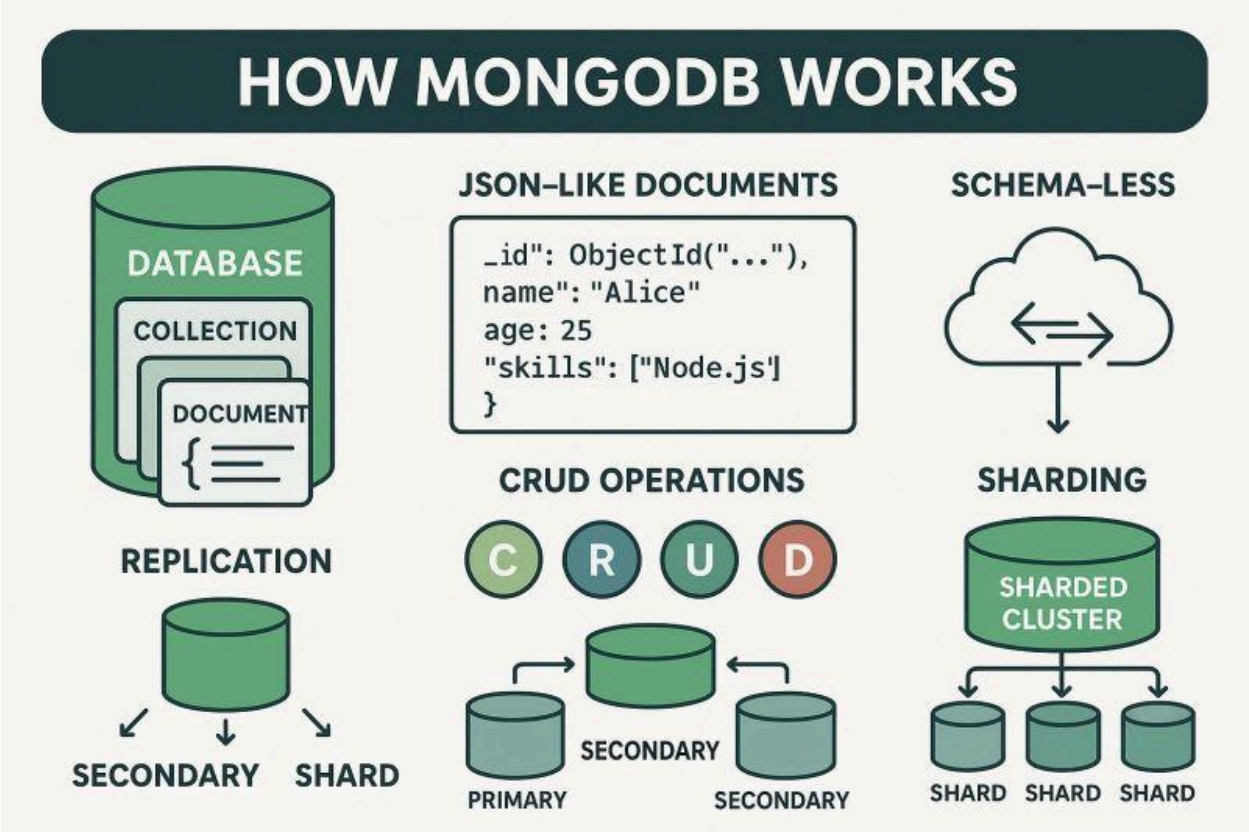
Traditional relational databases store data in tables consisting of rows and columns. Examples include MySQL, Oracle, and PostgreSQL.

What is NoSQL?

NoSQL stands for “Not Only SQL.” It refers to databases designed to store and process data using models other than, or in addition to, traditional relational tables.

NoSQL databases are useful when applications require flexible data structures, large-scale data processing, or distributed storage.

Types of NoSQL databases

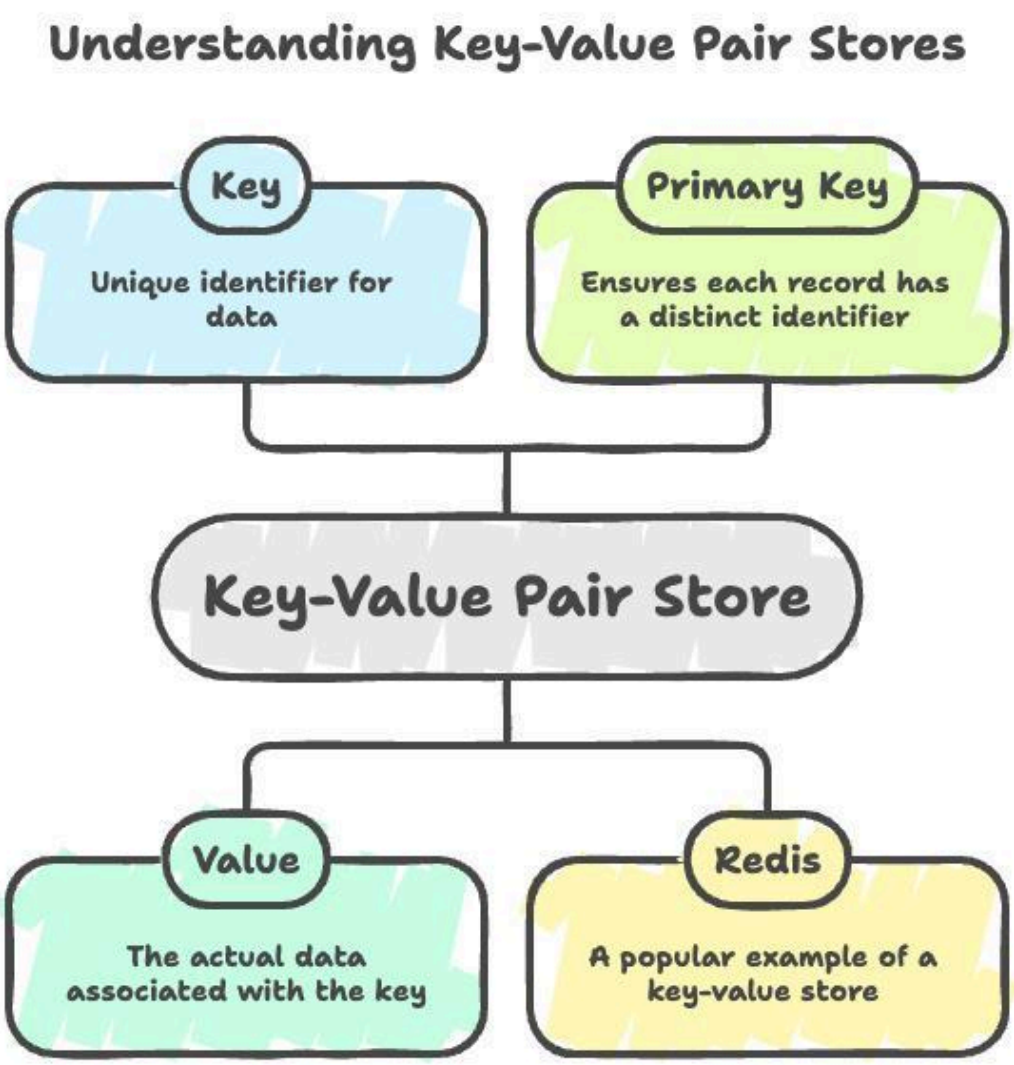


1. Document databases

Store data as documents containing fields and values.

Examples: MongoDB, CouchDB.

Use case: Student records, product catalogs.

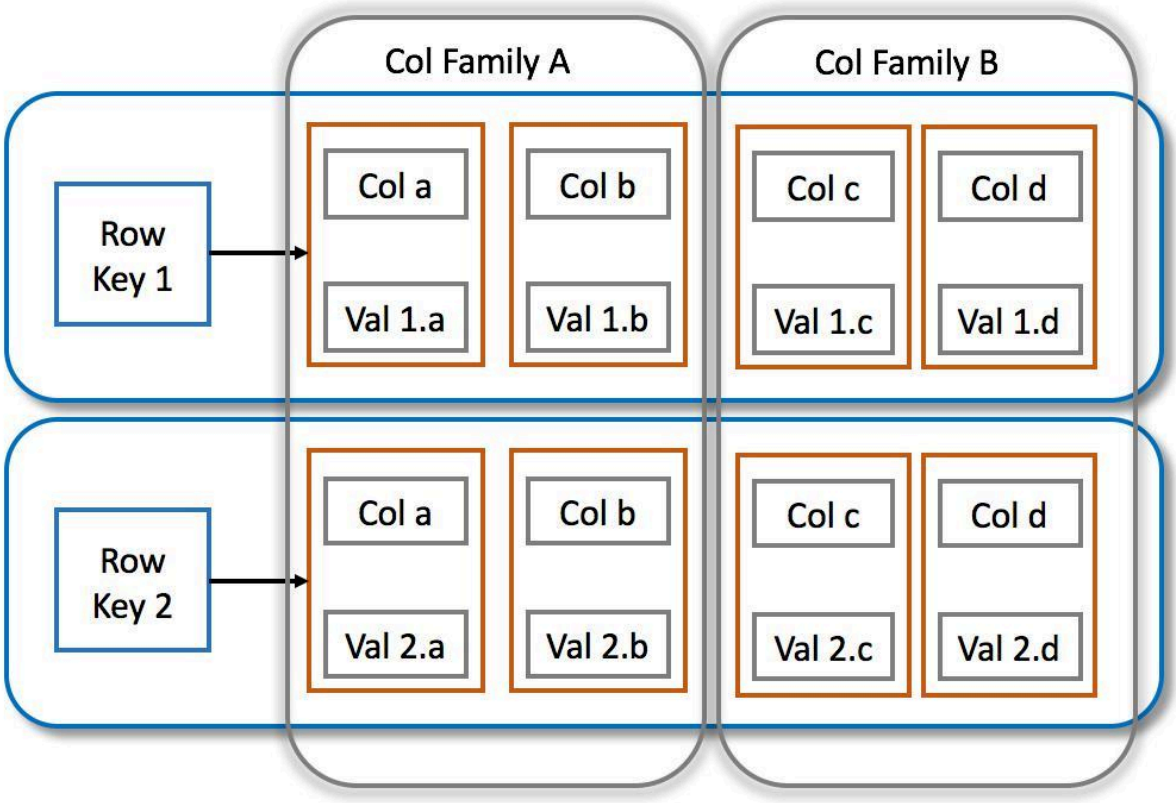


2. Key-value databases

Store data as a key associated with a value.

Examples: Redis, Amazon DynamoDB (supports multiple data models).

Use case: Caching, session management.

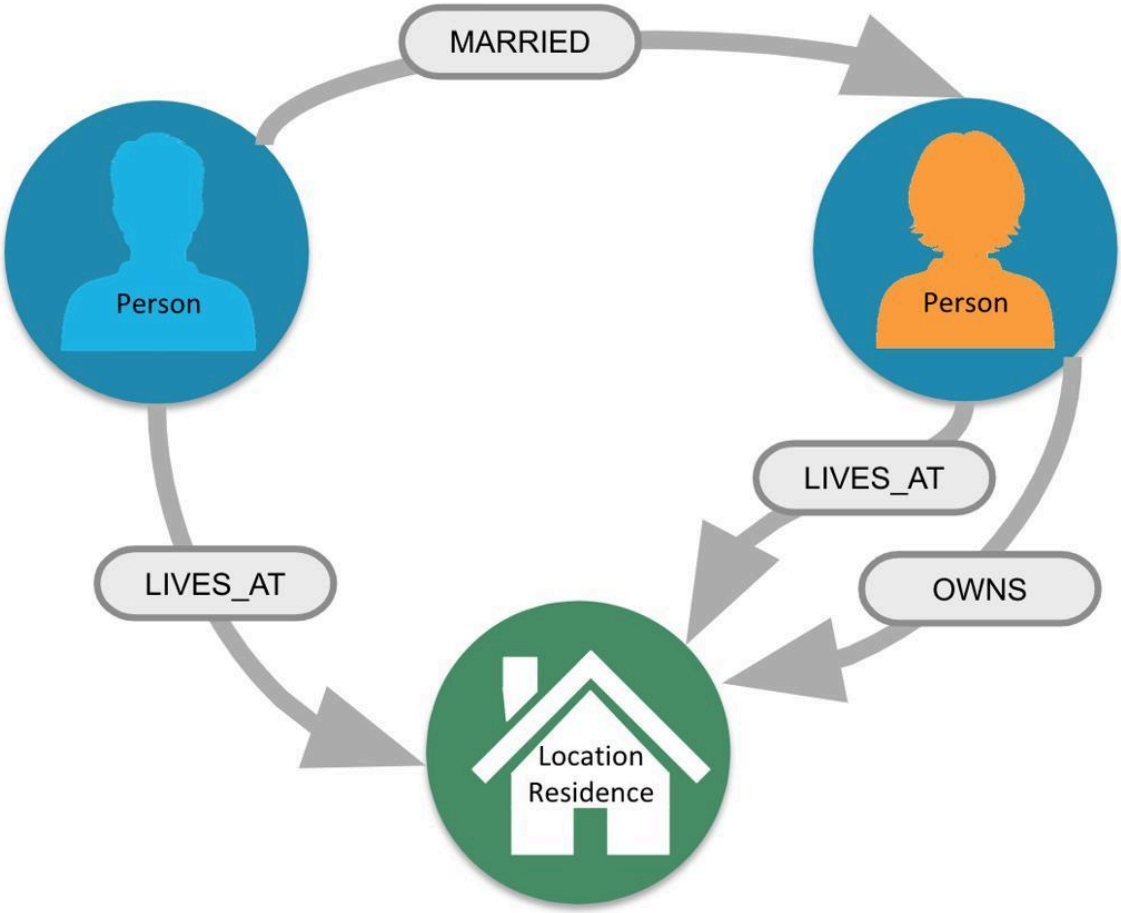


3. Wide-column databases

Organize data into column families for distributed storage.

Examples: Apache Cassandra, HBase.

Use case: Large-scale logs and time-series data.



4. Graph databases

Store data as nodes and relationships.

Examples: Neo4j, Amazon Neptune.

Use case: Social networks, recommendation systems.

What is MongoDB?

MongoDB is a document-oriented NoSQL database that stores data in BSON documents. Documents are organized into collections.

Example hierarchy:

MongoDB Database

Collection — `students`

Document 1

Student A

Document 2

Student B

Document 3

Student C

Example of a MongoDB document:

```
{
 _id: 101,
 name: "Rahul",
 age: 21,
 branch: "CSE",
 marks: 85
}
```

Here, `_id` uniquely identifies the document within its collection.

Features of MongoDB

1. Flexible schema: Documents in one collection can have different fields.
2. High scalability: Supports horizontal scaling using sharding.
3. High availability: Replica sets provide redundancy and failover.
4. Rich queries: Supports filtering, sorting, and aggregation.
5. Embedded documents: Related data can be stored inside a document.
6. Indexing: Indexes help improve query performance.

SQL vs NoSQL: Comparison

Feature	Relational database	MongoDB
Data model	Tables	Collections and documents
Structure	Rows and columns	Fields and values
Schema	Usually predefined	Flexible, optionally validated
Relationships	Foreign keys and joins	References and embedded documents
Query language	SQL	MongoDB Query API
Scaling	Often vertical; horizontal options exist	Horizontal scaling supported
Example	MySQL	MongoDB

Important exam point: NoSQL does not mean that a database has no structure or cannot support transactions. MongoDB supports schema validation and multi-document transactions.

2. JSON and BSON Data Formats

What is JSON?

JSON stands for JavaScript Object Notation. It is a text-based format used to represent and exchange structured data.

Example:

```
{
  "name": "Rahul",
  "age": 21,
  "isStudent": true,
  "skills": ["Java", "Python"]
}
```

JSON is human-readable and commonly used in web APIs and data exchange.

What is BSON?

BSON stands for Binary JSON. It is a binary-encoded document format used by MongoDB to store documents.

BSON represents JSON-like objects and supports additional data types, such as dates, ObjectId, and binary data.

Difference between JSON and BSON

Feature	JSON	BSON
Full form	JavaScript Object Notation	Binary JSON
Format	Text-based	Binary-encoded
Readability	Easy for humans to read	Not directly human-readable

Data types	Basic types such as strings, numbers, booleans, arrays and objects	Includes additional types such as Date, ObjectId, and binary data
Data size	Depends on textual representation	Depends on encoded types and field names; may be larger or smaller
Processing	Requires text parsing	Supports efficient typed decoding and traversal
Main use	Data exchange and APIs	MongoDB storage and communication

Example: JSON vs BSON

JSON representation:

```
{
 "name": "Rahul",
 "age": 21,
 "dob": "2005-02-10"
}
```

In JSON, the date shown above is a string.

A MongoDB document can instead store a real date value:

```
{
 name: "Rahul",
 age: 21,
 dob: ISODate("2005-02-10T00:00:00Z")
```

}

In the MongoDB shell, `ISODate()` represents a BSON Date value.

Conclusion: JSON is convenient for reading and exchanging data, whereas BSON is designed for storing and processing typed documents in MongoDB. BSON is not simply compressed JSON.

3. Working with MongoDB Data Types

MongoDB supports several BSON data types. Understanding them is important when designing documents and writing queries.

Important MongoDB data types

Data type	Description	Example
String	Stores text	`"name": "Rahul"`
Double	Stores floating-point numbers	`"cgpa": 8.5`
Int32	Stores 32-bit integers	`"age": NumberInt(21)`
Int64	Stores 64-bit integers	`"count": NumberLong(5000000000)`
Boolean	Stores true or false	`"passed": true`

Array	Stores a list of values	`"skills": ["Java", "Python"]`
Object / Document	Stores nested fields	`"address": {city: "Hyderabad"}`
ObjectId	Commonly used as a unique document identifier	`ObjectId("...")`
Date	Stores date and time in milliseconds since the Unix epoch	`ISODate("2026-01-01T00:00:00Z")`
Null	Represents a null value	`"middleName": null`
Decimal128	Stores decimal values with high precision	`NumberDecimal("99.95")`
Binary	Stores binary data	`BinData(...)`
Timestamp	Stores a special timestamp value	`Timestamp(...)`

Regular Expression	Stores a pattern for matching text	`/^Rah/`

Note: The MongoDB shell can automatically infer certain numeric types from literals. Constructors such as `NumberInt()`, `NumberLong()`, and `NumberDecimal()` make the intended type explicit.

Example of a document using multiple data types

```
{
  _id: ObjectId(),
  name: "Ananya",
  age: NumberInt(21),
  cgpa: 8.75,
  passed: true,
  skills: ["Java", "MongoDB", "Python"],
  address: {
    city: "Hyderabad",
    state: "Telangana"
  },
  joiningDate: ISODate("2026-01-10T00:00:00Z"),
  scholarship: NumberDecimal("15000.50"),
  middleName: null
}
```

Explanation:

- `name` is a string.
- `age` is a 32-bit integer.
- `cgpa` is a floating-point number.
- `skills` is an array.
- `address` is an embedded document.
- `joiningDate` is a BSON Date.
- `scholarship` is a decimal value.

- `middleName` is explicitly null.

Accessing nested fields and arrays

MongoDB uses dot notation to access nested fields.

db.students.find({ "address.city": "Hyderabad" })

This finds students whose city is Hyderabad.

To find students who have MongoDB as a skill:

db.students.find({ skills: "MongoDB" })

4. CRUD Operations in MongoDB

CRUD stands for:

- C — Create: Insert new documents.
- R — Read: Retrieve documents.
- U — Update: Modify existing documents.
- D — Delete: Remove documents.

The following examples use a student database and can be executed in the MongoDB shell, `mongosh`.

Step 1: Create a database and collection

use collegeDB

db.createCollection("students")

Explanation:

- `use collegeDB` switches to the database named `collegeDB`, creating it when data is first stored if it does not already exist.
- `db.createCollection("students")` explicitly creates a collection named `students`.

MongoDB can also create a collection automatically when data is first inserted into it.

A. Create operation — Insert documents

The `insertOne()` method inserts one document.

```
db.students.insertOne({
  rollNo: 101,
  name: "Rahul",
  age: 21,
  branch: "CSE",
  marks: 85
})
```

Expected result: An acknowledgment and the inserted document's `_id`.

To insert multiple documents, use `insertMany()`.

```
db.students.insertMany([
  {
    rollNo: 102,
    name: "Priya",
    age: 20,
    branch: "ECE",
    marks: 90
  },
  {
    rollNo: 103,
    name: "Amit",
    age: 21,
    branch: "CSE",
    marks: 78
  },
  {
    rollNo: 104,
```

```
    name: "Sneha",
    age: 22,
    branch: "IT",
    marks: 92
  }
])
```

This inserts three student documents.

Syntax:

```
db.collectionName.insertOne(document)
db.collectionName.insertMany([document1, document2])
```

B. Read operation — Retrieve documents

The `find()` method retrieves documents matching a condition. The `findOne()` method returns one matching document.

1. Display all students

```
db.students.find()
```

2. Find a student by roll number

```
db.students.findOne({ rollNo: 101 })
```

3. Find students from CSE

```
db.students.find({ branch: "CSE" })
```

4. Find students with marks greater than 80

```
db.students.find({ marks: { $gt: 80 } })
```

5. Find students with marks between 80 and 90, inclusive

```
db.students.find({
  marks: { $gte: 80, $lte: 90 }
})
```

6. Display only name and marks

```
db.students.find(
  {},
  { _id: 0, name: 1, marks: 1 }
)
```

Here, `1` includes a field and `0` excludes a field. By default, `_id` is included unless explicitly excluded.

Common query operators

Operator	Meaning	Example
`$eq`	Equal to	`{marks: {$eq: 85}}`
`$gt`	Greater than	`{marks: {$gt: 80}}`
`$lt`	Less than	`{marks: {$lt: 90}}`
`$gte`	Greater than or equal to	`{age: {$gte: 21}}`
`$lte`	Less than or equal to	`{marks: {$lte: 90}}`
`$ne`	Not equal to	`{branch: {$ne: "CSE"}}`

`$in`	Matches any listed value	`{branch: {$in: ["CSE", "IT"]}}`

C. Update operation — Modify documents

The `updateOne()` method updates the first matching document. The `updateMany()` method updates all matching documents.

1. Update one student's marks

```
db.students.updateOne(
  { rollNo: 101 },
  { $set: { marks: 95 } }
)
```

This changes Rahul's marks to 95.

2. Increase marks by 5

```
db.students.updateOne(
  { rollNo: 103 },
  { $inc: { marks: 5 } }
)
```

The `$inc` operator increments a numeric field by the specified value.

3. Add a new field

```
db.students.updateOne(
  { rollNo: 101 },
  { $set: { city: "Hyderabad" } }
)
```

This adds the `city` field if it does not exist or updates it if it already exists.

4. Update all CSE students

```
db.students.updateMany(
  { branch: "CSE" },
  { $set: { department: "Computer Science" } }
)
```

This adds or updates the `department` field for every matching CSE student.

Important update operators

Operator	Purpose
`$set`	Sets a field's value
`$inc`	Increments a numeric value
`$unset`	Removes a field
`$push`	Adds an element to an array
`$addToSet`	Adds an element only if it is not already present
`$pull`	Removes matching elements from an array

Example of adding a skill:

```
db.students.updateOne(
  { rollNo: 101 },
  { $push: { skills: "Python" } }
)
```

D. Delete operation — Remove documents

The `deleteOne()` method removes the first matching document, while `deleteMany()` removes all matching documents.

1. Delete one student

```
db.students.deleteOne({ rollNo: 104 })
```

2. Delete all students with marks below 40

```
db.students.deleteMany({
  marks: { $lt: 40 }
})
```

3. Delete all documents in the collection

```
db.students.deleteMany({})
```

This removes all documents but leaves the collection itself intact.

Be careful with `deleteMany({})`, as an empty filter matches every document.

CRUD operations summary

Operation	Method	Purpose
Create	`insertOne()`, `insertMany()`	Add documents

Read	`find()`, `findOne()`	Retrieve documents
Update	`updateOne()`, `updateMany()`	Modify documents
Delete	`deleteOne()`, `deleteMany()`	Remove documents

5. Schema Design in MongoDB

What is a schema?

A schema defines the structure of data, including fields, data types, relationships, and validation rules.

In relational databases, schemas usually define tables and columns in advance. MongoDB has a flexible schema, which means documents within the same collection do not have to contain exactly the same fields.

However, schema design is still important for data consistency, query performance, and efficient storage.

Example of a flexible schema

Two documents in the same `students` collection can have different fields:

```
{
 rollNo: 101,
 name: "Rahul",
 branch: "CSE",
 marks: 85
```

```
}
{
  rollNo: 102,
  name: "Priya",
  branch: "ECE",
  marks: 90,
  scholarship: 15000
}
```

The second document has an additional `scholarship` field. MongoDB permits this by default, unless validation rules restrict the structure.

Two major schema design approaches

1. Embedded documents

In embedding, related information is stored inside a single document.

Example: A student and their address.

```
{
  rollNo: 101,
  name: "Rahul",
  address: {
    street: "Madhapur",
    city: "Hyderabad",
    state: "Telangana"
  }
}
```

Advantages:

- Related data can be retrieved in a single query.
- Fewer separate database operations may be required.
- Useful for data that belongs together and is usually accessed together.

Disadvantages:

- Repeated embedded data can increase storage.
- Frequently changing shared information can become difficult to maintain.
- Documents have a maximum BSON size of 16 MiB.

2. References

In referencing, related information is stored in separate documents, and one document contains an identifier that points to another.

Example:

Student collection:

```
{
 _id: 101,
 name: "Rahul",
 departmentId: 501
}
```

Department collection:

```
{
 _id: 501,
 departmentName: "Computer Science",
 hod: "Dr. Kumar"
}
```

Here, `departmentId` refers to the department document's `_id`.

Advantages:

- Reduces duplication of shared information.
- Suitable for large or independently managed related records.
- Easier to update shared information in one place.

Disadvantages:

- Additional queries or `$lookup` aggregation operations may be required.
- Application logic can become more complex.

Embedding vs referencing

Feature	Embedding	Referencing
Storage	Related data in one document	Related data in separate documents
Retrieval	Often a single query	May require multiple queries or `$lookup`
Duplication	Can be higher	Often lower
Best suited for	Small, closely related data	Large, shared, or independently updated data
Example	Student with address	Student linked to department

Schema design principles

1. Design around queries: Store data according to how the application reads and updates it.
2. Embed data accessed together: For example, a user's basic profile and address.
3. Use references for shared data: For example, many students belonging to one department.
4. Avoid unnecessary duplication: Especially when duplicated values change frequently.
5. Use indexes: Index frequently queried fields to improve performance.
6. Apply validation: Ensure required fields and appropriate data types are present.

7. Consider document size and growth: Avoid unbounded arrays and excessively large documents.

Schema validation example

MongoDB allows validation rules using JSON Schema.

```
db.createCollection("examResults", {
  validator: {
    $jsonSchema: {
      bsonType: "object",
      required: ["rollNo", "name", "marks"],
      properties: {
        rollNo: {
          bsonType: "int"
        },
        name: {
          bsonType: "string"
        },
        marks: {
          bsonType: "int",
          minimum: 0,
          maximum: 100
        }
      }
    }
  }
})
```

This creates a collection that requires `rollNo`, `name`, and `marks`, with specified types and marks between 0 and 100. Documents that violate the validation rules are rejected under the default error behavior.

6. Complete Practical Example: Student Management System

The following program combines schema design and CRUD operations into one practical example.

Step 1: Select a database

```
use collegeDB
```

Step 2: Insert sample data

```
db.students.insertMany([
  {
    rollNo: 101,
    name: "Rahul",
    branch: "CSE",
    marks: 85,
    skills: ["Java", "Python"]
  },
  {
    rollNo: 102,
    name: "Priya",
    branch: "ECE",
    marks: 92,
    skills: ["C", "MATLAB"]
  },
  {
    rollNo: 103,
    name: "Amit",
    branch: "CSE",
    marks: 74,
    skills: ["Java", "MongoDB"]
  }
])
```

Step 3: Retrieve CSE students

```
db.students.find({ branch: "CSE" })
```

Expected matching students: Rahul and Amit.

Step 4: Find students scoring above 80

```
db.students.find({
  marks: { $gt: 80 }
})
```

Expected matching students: Rahul and Priya.

Step 5: Update Amit's marks

```
db.students.updateOne(
  { rollNo: 103 },
  { $set: { marks: 82 } }
)
```

Step 6: Add a skill to Rahul's document

```
db.students.updateOne(
  { rollNo: 101 },
  { $addToSet: { skills: "MongoDB" } }
)
```

`$addToSet` adds the skill only if it is not already present in the array.

Step 7: Delete a student

```
db.students.deleteOne({ rollNo: 102 })
```

This deletes Priya's document.

Step 8: Verify the remaining documents

```
db.students.find()
```

The collection should now contain Rahul and Amit, with their updated marks and skills.

7. Advantages and Limitations of MongoDB

Advantages

- Flexible document structure.

- Suitable for semi-structured and evolving data.
- Supports indexing and aggregation.
- Supports replication and horizontal scaling.
- Allows embedded documents and arrays.
- Supports transactions, including multi-document transactions.

Limitations

- Poor schema planning can lead to inconsistent data.
- Document duplication can increase storage and update complexity.
- Complex relationships may require additional queries or aggregation.
- Large or unbounded embedded arrays can create performance problems.
- Indexes consume additional storage and slow some write operations.
- It is not automatically faster than relational databases; performance depends on the workload and design.

8. Important Exam Questions for B.Tech 3rd Year

1. Define NoSQL. Explain the types of NoSQL databases.

2. Compare JSON and BSON.

3. Explain MongoDB data types with examples.

4. Explain CRUD operations in MongoDB with syntax and examples.

5. Explain embedding and referencing in MongoDB schema design.

6. Write MongoDB queries to insert, retrieve, update and delete student records.

7. Explain MongoDB query and update operators.

8. Explain schema validation in MongoDB.

9. Quick Revision

MongoDB

A document-oriented NoSQL database that organizes documents into collections.

JSON vs BSON

JSON is text-based; BSON is binary-encoded and supports additional data types.

CRUD

Create: `insertOne()` · Read: `find()` · Update: `updateOne()` · Delete: `deleteOne()`

Schema design

Use embedding for closely related data and referencing for shared or independently managed data.